

Danmarks
Tekniske
Universitet



Videnskabelig maskinlæring til simulering af plasmaranddynamikker

AUTHOR

Andreas Bjarnastein Antoft - s224041

June 17, 2026

Contents

1	Introduction	1
2	Metode	4
2.1	Approximation af PDE'er med neurale netværk	4
2.1.1	Hard constraints for randbetingelser	4
2.1.2	Physics-informed neural networks og soft constraints	5
2.2	Fysikken bag Hesel modellen	6
2.2.1	Braginskii's fluidligninger	6
2.2.2	Hesel modellen	10
2.3	Numeriske metoder	15
2.3.1	Rumlig diskretisering	15
2.3.2	Numeriske differentialoperatorer	15
2.3.3	Tidsintegration	16
2.4	Tilgang til eksperiment	17
2.4.1	Opstilling af eksperimenter	17
2.4.2	Valg af model arkitektur	18
3	Implementering	19
3.1	Overordnet struktur	19
3.1.1	Pipeline for modellen	19
3.1.2	Sammenhæng mellem simulation, data og model	20
4	Data og eksperiment	23
4.1	Opsætning og generering af data	23
4.1.1	BOUT opsætning	23
4.1.2	Data generering	23
4.1.3	Data visualisering	24
5	Resultater	25
6	Konklusion og diskussion	26
7	Fremtidigt arbejde	27
	List of Figures	I
	List of Tables	II
	Nomenclature	III
	References	IV
8	Appendix A	V
8.1	Input fil for konfiguration af simulering	V

8.2	Opsætning af BOUT++ og BOUT-HESEL	XIII
9	Appendix B	XV
9.1	Anvendelse af Boltzmann ligning	XV

1 Introduction

Verdens energiforbrug forventes at stige markant i de kommende årtier, samtidig med at behovet for stabile energikilder med lave CO₂ udledninger bliver stadig mere presserende. I den sammenhæng fremstår fusionsenergi som en særligt attraktiv mulighed, fordi den i princippet kan levere store mængder energi med et relativt begrænset brændselsforbrug og uden de samme langsigtede affaldsproblemer som konventionel fission [1]. Potentialer er derfor betydeligt, men vejen fra fysisk mulighed til teknologisk realisering er lang. En af de centrale udfordringer er, at plasmaet i en fusionsreaktor er et meget komplekst system, hvor små ændringer i lokale forhold kan føre til markante forskelle i transport, stabilitet og energitab.

I denne rapport er fokus rettet mod tokamakken, som er en af de mest undersøgte konfigurationer inden for magnetisk indesluttet fusion. I en tokamak holdes et varmt, ioniseret plasma på plads af stærke magnetfelter, så partiklerne ikke kommer i direkte kontakt med reaktorens vægge. Særligt interessant er plasmaets randregion, altså overgangen mellem det godt indesluttet plasma og scrape-off layer (SOL), hvor feltlinjerne ikke længere er lukkede. Det er netop i denne randzone, at turbulens, filamentstrukturer og tværgående transport i høj grad bestemmer, hvor effektiv indeslutningen er, og hvor store belastninger væggene i reaktoren udsættes for. Hvis disse processer ikke forstås og beskrives tilstrækkeligt præcist, bliver det vanskeligt at optimere energibalancen i reaktoren [2, 3].

Randplasmaets dynamik er imidlertid vanskelig at analysere direkte. Fysikken beskrives gennem koblede, ikke lineære partielle differentialligninger, hvor tæthed, tryk, potentiale og strømning påvirker hinanden gensidigt. Selv reducerede modeller rummer derfor et komplekst samspil mellem transport, dissipation og instabiliteter. I dette projekt anvendes Hot edge-SOL-elctrostatic (HESEL) modellen, som er en reduceret drift fluid model udviklet til at beskrive turbulens og transport i plasmaets rand og i SOL regionen [4, 3]. Modellen er fysisk konsistent nok til at repræsentere centrale mekanismer i randplasmaet, men samtidig tilstrækkeligt reduceret til at kunne simuleres numerisk i praksis. Netop derfor er HESEL modellen et godt udgangspunkt for at undersøge, hvordan moderne datadrevne metoder kan kombineres med klassisk plasmafysisk modellering.

Den traditionelle tilgang til HESEL og lignende modeller er numerisk simulering, eksempelvis ved finite difference diskretisering i rum og tidsintegration af det resulterende ligningssystem. Denne tilgang er veletableret og giver adgang til detaljerede løsninger, men den er også beregningstung og kan være numerisk ustabil. Dertil kommer, at plasmaets raddynamik er stærkt ikke lineær og følsom over for begyndelsesbetingelser og parameterændringer. Hvis man ønsker at sammenligne mange scenarier, udforske store parameterum eller bruge modellen i en lille analyse, bliver fulde simuleringer hurtigt en flaskehals. Problemet er derfor ikke alene at kunne simulere systemet korrekt, men også at kunne gøre det effektivt og på en måde, der bevarer den væsentlige fysik.

Her bliver Scientific Machine Learning (SciML) interessant. SciML betegner metoder, hvor maskinlæring kombineres med kendt struktur fra naturvidenskabelige modeller, eksempelvis differentialligninger, symmetrier eller bevarelseslove. I stedet for at betragte data og fysik som to adskilte kilder til viden forsøger SciML at udnytte begge dele samtidig. For

plasmafysik er det særligt lovende, fordi mange problemer netop er kendetegnet ved dyre simuleringer, begrænsede datamængder og stærke fysiske begrænsninger, som modellerne bør respektere. En vellykket SciML model kan derfor potentielt fungere som en hurtigere surrogatmodel, der enten helt eller delvist kan erstatte tunge simuleringer i udvalgte arbejdsområder.

Projektets motivation er den en metodisk motivation: hvis SciML kan anvendes meningsfuldt på HESEL modellen, peger det mod en mere fleksibel måde at arbejde med komplekse plasma modeller på.

Projektet placerer sig derfor i krydsfeltet mellem anvendt matematik, numerisk analyse, plasmafysik og maskinlæring, og netop dette tværfaglige spændingsfelt gør undersøgelsen fagligt relevant.

Formålet med rapporten er at undersøge, hvordan SciML kan anvendes til modellering af raddynamik i plasma med udgangspunkt i HESEL modellen, og i hvilken grad sådanne modeller kan supplere eller effektivisere klassiske numeriske simuleringer. Udgangspunktet er ikke, at maskinlæring ukritisk skal erstatte fysikbaserede metoder, men snarere at afklare, hvornår og hvordan en fysikinformeret læringsmodel kan være et nyttigt redskab. Derfor lægges der vægt på både fysisk fortolkning, numerisk sammenlignelighed og metodiske begrænsninger.

På baggrund af projektplanen undersøges følgende spørgsmål:

1. Hvordan kan SciML, særligt fysikinformerede neurale netværk, anvendes til modellering af HESEL-ligningssystemet, og hvordan sammenlignes løsningerne med klassiske numeriske simuleringer?
2. Hvilken betydning har det for modellens kvalitet, om man træner på de enkelte ligninger hver for sig eller på det fulde koblede ligningssystem, og hvordan påvirker dataopløsningen resultaterne?
3. I hvor høj grad kan modellen generalisere til ukendte punkter, tidsskridt og begyndelsesbetingelser, og hvordan hænger denne generalisering sammen med valg af hyperparametre og træningsopsætning?
4. Hvor meget kan modellen effektiviseres, før tabet i præcision bliver væsentligt, og hvordan bør beregningstid, hukommelsesforbrug og nøjagtighed afvejes i praksis?

Disse spørgsmål er valgt, fordi de tilsammen dækker både den fysiske og den beregningsmæssige side af problemet. Det er ikke tilstrækkeligt, at en model blot kan reproducere enkelte løsninger; den skal også være robust, fortolkbart og anvendelig i situationer, hvor man ønsker mere end ren interpolation. Derfor er fokus ikke alene på fejlmål, men også på modellens anvendelighed som repræsentation af dynamikken i et ikke-lineært system. I den forstand handler projektet både om modelleringsevne og om videnskabelig troværdighed.

Rapporten har samtidig et tydeligt læringsperspektiv, selv om dette ikke er hovedformålet i selve analysen. Arbejdet forudsætter en praktisk forståelse af numerisk løsning af PDE-systemer, grundlæggende plasmafysik i randregionen og udvikling samt evaluering af SciML-modeller på dynamiske systemer. Desuden indgår sammenligningen mellem simuleringer og lærte modeller som et centralt metodisk element. Læringsmålene fungerer derfor

som en bagvedliggende ramme for projektet: de angiver, hvilke kompetencer der skal bringes i spil for at kunne besvare forskningsspørgsmålene på en fagligt meningsfuld måde.

Resten af rapporten er struktureret som følger. Først præsenteres den fysiske baggrund for HESEL-modellen og de numeriske metoder, der anvendes til simulering af ligningssystemet. Herefter beskrives implementeringen af eksperimenterne og den måde, data genereres og behandles på. Dernæst analyseres resultaterne fra både de klassiske simuleringer og de anvendte SciML-metoder med henblik på at vurdere nøjagtighed, generalisering og effektivitet. Afslutningsvis diskuteres projektets væsentligste resultater, begrænsninger og perspektiver for fremtidigt arbejde.

2 Metode

Her gennemgås det teori og metode tilgang

2.1 Approximation af PDE'er med neurale netværk

Neurale netværk kan bruges til at approksimere løsningen til en differentiaalligning ved at repræsentere den ukendte løsning som en differentiabel funktion med et sæt justerbare parametre. I stedet for først at diskretisere ligningen på et fast gitter kan man indsætte den approksimerede løsning direkte i den kontinuerte ligning og derefter justere parametrene, så funktionen både opfylder differentiaalligningen og de tilhørende rand- eller initialbetingelser [5].

For et generelt PDE-problem kan løsningen skrives som

$$\mathcal{G}(t, x, y, u, \partial_t u, \nabla u, \nabla^2 u) = 0, \quad (t, x, y) \in [0, T] \times \Omega, \quad (1)$$

med randbetingelser

$$\mathcal{B}[u] = g \quad \text{på } \partial\Omega \quad (2)$$

og eventuelt en initialbetingelse

$$u(0, x, y) = u_0(x, y). \quad (3)$$

Metoden erstatter den ukendte løsning med en approksimation $u_\theta(t, x, y)$, hvor θ samler de frie parametre. Ved at indsætte u_θ i ligning (1) fås residualet

$$r_\theta(t, x, y) = \mathcal{G}(t, x, y, u_\theta, \partial_t u_\theta, \nabla u_\theta, \nabla^2 u_\theta). \quad (4)$$

Hvis u_θ var den eksakte løsning, ville residualet være nul overalt i domænet. Træningen består derfor i at vælge θ , så residualet bliver lille i et antal kollokationspunkter i domænet.

2.1.1 Hard constraints for randbetingelser

En klassisk tilgang er at indbygge rand- og initialbetingelser direkte i ansatsen for løsningen. I formuleringen fra [5] skrives den approksimerede løsning som

$$u_\theta(t, x, y) = A(t, x, y) + M(t, x, y)N_\theta(t, x, y), \quad (5)$$

hvor A vælges, så den alene opfylder rand- og initialbetingelserne, mens maskefunktionen M konstrueres, så den forsvinder på de dele af randen, hvor Dirichlet-betingelser pålægges. Dermed påvirker korrektionsleddet MN_θ ikke de påtvungne randværdier. Ved blandede randbetingelser kan M tilsvarende vælges, så også den normale afledte forsvinder på de relevante randsegmenter.

For en elliptisk ligning, eksempelvis Poisson-ligningen

$$\nabla^2 u(x, y) = f(x, y), \quad (x, y) \in \Omega,$$

kan residualet evalueres direkte fra de rumlige afledte af u_θ . For tidsafhængige PDE'er behandles tiden blot som endnu en inputvariabel, så både $\partial_t u_\theta$ og de rumlige afledte indgår i residualet. For et koblet PDE-system kan approksimationen tilsvarende skrives som en vektor

$$\mathbf{u}_\theta(t, x, y) \in \mathbb{R}^m,$$

hvor alle felter trænes samtidig ved at minimere residualerne for hele systemet.

Når randbetingelserne er indbygget på denne måde, behøver tabsfunktionen kun at håndhæve selve differentialligningen, og en typisk tabsfunktion kan derfor skrives som

$$\mathcal{L}(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} |r_\theta(t_i, x_i, y_i)|^2, \quad (6)$$

hvor $\{(t_i, x_i, y_i)\}_{i=1}^{N_r}$ er kollokationspunkter i domænet. Randbetingelserne håndhæves her som hard constraints, fordi de opfyldes præcist ved konstruktion af ansatsen.

2.1.2 Physics-informed neural networks og soft constraints

I physics-informed neural networks (PINNs) bruges den samme grundidé, nemlig at approksimere den ukendte løsning med et neuralt netværk og evaluere differentialligningen gennem et residual. I stedet for at indbygge rand- og initialbetingelser eksplicit i ansatsen vælges der dog ofte en mere direkte parameterisering af løsningen, og betingelserne håndhæves i tabsfunktionen. Denne formulering gennemgås i *Physics Informed Deep Learning* af Raissi, Perdikaris og Karniadakis [6].

For et tidsafhængigt PDE-problem på formen

$$\partial_t u + \mathcal{N}[u] = 0, \quad (7)$$

hvor $\mathcal{N}$ er en ikke-lineær differentialoperator, introduceres både en approksimation $u_\theta(t, x, y)$ og et tilhørende fysik-residual

$$f_\theta(t, x, y) = \partial_t u_\theta(t, x, y) + \mathcal{N}[u_\theta](t, x, y). \quad (8)$$

De samme parametre θ indgår i både u_θ og f_θ , så optimeringen samtidig former løsningen og dens afledte egenskaber.

I denne tilgang opdeles tabsfunktionen i et dataled og et fysikled,

$$\mathcal{L}(\theta) = \mathcal{L}_u + \mathcal{L}_f, \quad (9)$$

hvor

$$\mathcal{L}_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_\theta(t_u^i, x_u^i, y_u^i) - u^i|^2 \quad (10)$$

måler afvigelsen fra kendte initial- og randdata, mens

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f_\theta(t_f^i, x_f^i, y_f^i)|^2 \quad (11)$$

måler, hvor godt differentiaalligningen opfyldes i et antal kollokationspunkter i domænets indre. Her angiver $\{(t_u^i, x_u^i, y_u^i), u^i\}_{i=1}^{N_u}$ de punkter, hvor løsningens værdi er kendt fra initial- eller randbetingelser, mens $\{(t_f^i, x_f^i, y_f^i)\}_{i=1}^{N_f}$ er de punkter, hvor fysikken håndhæves gennem residualet.

Denne formulering svarer til soft constraints, fordi rand- og initialbetingelser ikke nødvendigvis opfyldes præcist i hvert punkt, men i stedet gøres dyre at bryde gennem tabsfunktionen. Til gengæld er metoden fleksibel, fordi samme grundstruktur kan anvendes til mange forskellige typer PDE'er uden at skulle konstruere en særlig ansats for hver ny kombination af randbetingelser. Fysikleddet $\mathcal{L}_f$ virker samtidig som en regularisering, der begrænser løsningsrummet til funktioner, som i høj grad respekterer den underliggende differentiaalligning [6].

Forskellen mellem de to formuleringer er derfor primært, hvordan rand- og initialbetingelser håndhæves. Hard constraints bygger betingelserne ind i selve ansatsen og opfylder dem præcist, mens PINN-formuleringen med soft constraints håndhæver dem gennem optimeringen. I begge tilfælde er målet at finde en glat funktion, som approksimerer løsningen til PDE'en i hele domænet frem for kun i et endeligt antal gitterpunkter.

2.2 Fysikken bag Hesel modellen

HESEL modellen (Hot Edge SOL Electrostatic) er en reduceret plasma model udviklet til at beskrive turbulens og transportfænomener i plasmaets rand i en tokamak. Særligt fokuserer modellen på overgangsområdet mellem det indelukkede plasma og scrape off layer (SOL), hvor magnetfeltlinjerne ikke længere er lukkede. I dette område transporteres partikler og energi på tværs af magnetfeltlinjerne gennem turbulente strukturer, kaldet blobs. HESEL modellen er baseret på drift fluid approksimationen af Braginskii's plasma fluidligninger[4].

HESEL bliver reduceret, fra Braginskii's ligninger, ved at udnytte de karakteristiske tids og længdeskalaer i plasmaets rand, hvilket vedligeholder simulering fysisk realitet og numerisk stabilitet.

I dette afsnit gennemgås de grundlæggende trin i Braginskii's udledning, hvorefter den endelige HESEL model præsenteres. Formålet er ikke at give en fuldstændig fysisk redegørelse for HESEL modellen, men derimod at introducere de centrale idéer og ligninger, som modellen bygger på.

2.2.1 Braginskii's fluidligninger

Braginskii's plasma fluid ligninger er et sæt makroskopiske ligninger, der beskriver bevarelsen af partikler, impuls og energi for en plasmaart. Som udgangspunkt anvendes

Boltzmann ligningen, som beskriver udviklingen af plasmaets fordelingsfunktion i fase rummet. Faserummet består af både partiklernes position $\mathbf{r}$ og hastighed $\mathbf{v}$. For et kollisionsfrit system gælder

$$\frac{df_a}{dt} = 0.$$

Anvendes kædereolen i faserummet fås:

$$\frac{df_a}{dt} = \frac{\partial f_a}{\partial t} + \frac{dx_\beta}{dt} \frac{\partial f_a}{\partial x_\beta} + \frac{dv_\beta}{dt} \frac{\partial f_a}{\partial v_\beta}.$$

Herefter sættes:

$$\begin{aligned} \frac{dx_\beta}{dt} &= v_\beta, \\ \frac{dv_\beta}{dt} &= \frac{F_{a\beta}}{m_a} \end{aligned}$$

Når kollisioner inkluderes opnås Boltzmann ligningen

$$\frac{\partial f_a}{\partial t} + \frac{\partial}{\partial x_\beta} (v_\beta f_a) + \frac{\partial}{\partial v_\beta} \left(\frac{F_{a\beta}}{m_a} f_a \right) = C_a \quad (12)$$

Her er β indeks for koordinater i det fysiske rum. Det første led, $\partial_t f_a$, beskriver den lokale tidslige ændring af fordelingsfunktionen. Det andet led, $\partial_{x_\beta} (v_\beta f_a)$, beskriver transport i det fysiske rum. Det tredje led, $\partial_{v_\beta} \left(\frac{F_{a\beta}}{m_a} f_a \right)$, beskriver transport i hastighedsrummet. Højresiden C_a beskriver ændringen i fordelingsfunktionen som følge af kollisioner mellem partikler. Et lille eksempel på anvendelse af (12) kan ses i appendix B 9.1.

I ligningen indgår følgende størrelser:

$f_a = f_a(t, \mathbf{r}, \mathbf{v})$	fordelingsfunktion for partikelart a ,
$\mathbf{F}_a = e_a \mathbf{E} + \frac{e_a}{c} (\mathbf{v} \times \mathbf{B})$	Lorentzkraften på partikelart a ,
m_a	massen af partikelart a ,
e_a	ladningen af partikelart a ,
C_a	kollisionsleddet for partikelart a .

Her beskriver $f_a(t, \mathbf{r}, \mathbf{v}) d\mathbf{v} d\mathbf{r}$ tætheden af partikler af art a i det fysiske rum $d\mathbf{r}$ omkring punktet $\mathbf{r}$. I resten af udledningen undlades det at specificere hvilken partikelart, der er tale om på samme måde som Braginskii gør. Da kollisioner over alle hastigheder ikke ændrer det totale antal partikler gælder for kollisions termet.

$$\int C d\mathbf{v} = 0,$$

Herefter defineres følgende makroskopiske størrelser:

$$\begin{aligned} n(\mathbf{r}, t) &= \int f(\mathbf{r}, \mathbf{v}, t) d\mathbf{v} \\ V_a(\mathbf{r}, t) &= \frac{1}{n} \int \mathbf{v} f(\mathbf{r}, \mathbf{v}, t) d\mathbf{v} = \langle v \rangle \\ T(\mathbf{r}, t) &= \frac{m}{3} \langle (v - V)^2 \rangle \end{aligned}$$

Her er n , V og T makroskopiske størrelser for tætheden, middelhastigheden og temperaturen, for partikler i punktet $\mathbf{r}$ til tiden t . For at komme frem til Braginskii's 3 ligninger, der beskriver tæthed, impuls og energi, ganges (12) med 1, mv og $\frac{1}{2}mv^2$. Herefter integreres ligningerne over hele hastighedsrummet, hvorefter n , V og T indsættes i ligningerne. Herved fremkommer Braginskii's ligninger:

$$\frac{\partial n}{\partial t} + \nabla \cdot (n\mathbf{V}) = 0 \quad (13)$$

$$\frac{\partial}{\partial t} (mn\mathbf{V}) + \frac{\partial}{\partial x_\beta} (mn\langle vv_\beta \rangle) - en \left(E + \frac{1}{c} [\mathbf{V}\mathbf{B}] \right) = \int mvC d\mathbf{v} \quad (14)$$

$$\frac{\partial}{\partial t} \left(\frac{mn}{2} \langle v^2 \rangle \right) + \nabla \cdot \left(\frac{mn}{2} \langle v^2 \mathbf{v} \rangle \right) - en\mathbf{E}\mathbf{V} = \int \frac{mv^2}{2} C d\mathbf{v} \quad (15)$$

(Selve integrationen og udledningen af ligningerne er ikke udført i denne rapport.) Ligning (13) kaldes kontinuitetsligningen og beskriver bevarelsen af partikler. Ligningerne (14) og (15) kaldes henholdsvis impuls og energiligningen og beskriver bevarelsen af henholdsvis impuls og energi i plasmaet. (14) og (15) bliver yderligere reduceret.

For hastigheden af partikeltyper defineres middel hastighed $\mathbf{V}$ og tilfældig hastighed, $\mathbf{v}' = \mathbf{v} - \mathbf{V}$. Med dette udledes:

$$\begin{aligned} \langle v_\alpha v_\beta \rangle &= \langle (V_\alpha + v'_\alpha)(V_\beta + v'_\beta) \rangle \\ &= V_\alpha V_\beta + V_\alpha \langle v'_\beta \rangle + V_\beta \langle v'_\alpha \rangle + \langle v'_\alpha v'_\beta \rangle \end{aligned}$$

Da den tilfældige hastighed er defineret som afvigelsen fra middelhastigheden,

$$v'_\alpha = v_\alpha - V_\alpha$$

gælder

$$\langle v'_\alpha \rangle = \langle v'_\beta \rangle = 0$$

Dermed fås

$$\langle v_\alpha v_\beta \rangle = V_\alpha V_\beta + \langle v'_\alpha v'_\beta \rangle$$

For at simplificere Braginskii's impuls ligning (14) defineres følgende:

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + V_\beta \frac{\partial f}{\partial x_\beta} = \frac{\partial f}{\partial t} + (\mathbf{V} \nabla) f \quad (16)$$

$$p = \frac{mn \langle v'^2 \rangle}{3} = nT \quad (17)$$

$$\pi_{\alpha\beta} = nm \langle v'_\alpha v'_\beta - \frac{1}{3} v'^2 \delta_{\alpha\beta} \rangle \quad (18)$$

$$\mathbf{R} = \int m \mathbf{v}' C d\mathbf{v} \quad (19)$$

Operatoren (16) kaldes substantive afledte og er anvendt på funktionen f . Ligning (17) definerer det isotrope tryk, (18) den viskøse spændingstensor der beskriver afvigelser fra isotropt tryk, (19) beskriver netto momentumoverførsel som følge af kollisioner mellem partikler.

Braginskii's impuls ligning kan skrives på formen:

$$mn \frac{dV_a}{dt} = -\frac{\partial p}{\partial x_a} - \frac{\partial \pi_{a\beta}}{\partial x_\beta} + en \left(E_a + \frac{1}{c} [\mathbf{VB}]_a \right) + R_a \quad (20)$$

Venstresiden, af Braginskii's momentum ligning (20), repræsenterer ændringen i plasmaets momentum. Højresiden består af kræfter fra trykgradienter, viskøse spændinger, elektromagnetiske felter og kollisioner med andre partikler. Disse processer styrer transporten og udviklingen af plasmaets makroskopiske bevægelse.

Braginskii's energi ligning (15) er også ønsket at skrive på en mere kompakt form. Til dette udledes følgende størrelser:

$$\begin{aligned} \langle \frac{v^2}{2} v_\beta \rangle &= \frac{1}{2} V^2 V_\beta + V_\alpha \langle v'_\alpha v'_\beta \rangle + \frac{1}{2} \langle v'^2 \rangle V_\beta + \langle \frac{v'^2}{2} v'_\beta \rangle \\ &= \left(\frac{1}{2} V^2 + \frac{5P}{2mn} \right) V_\beta + \frac{1}{mn} V_\alpha \pi_{\alpha\beta} + \langle \frac{1}{2} v'^2 v'_\beta \rangle \end{aligned}$$

Ved at introducere følgende definitioner:

$$\begin{aligned} Q &= \int \frac{m}{2} v'^2 C d\mathbf{v} \\ \mathbf{q} &= \int \frac{m}{2} v'^2 \mathbf{v}' f d\mathbf{v} = \langle \frac{1}{2} m v'^2 \mathbf{v}' \rangle \end{aligned}$$

Hvor Q beskriver varme genereret af kollisioner mellem partikler. $\mathbf{q}$ er flux tætheden for

tilfældig transport. Herfra kan Braginskii's energi ligning komme på den kompakte form:

$$\frac{\partial}{\partial t} \left(\frac{nm}{2} V^2 + \frac{3}{2} nT \right) + \frac{\partial}{\partial x_\beta} \left(\left(\frac{mn}{2} V^2 + \frac{5}{2} nT \right) V_\beta + (\pi_{\alpha\beta} V_\alpha) + q_\beta \right) = en\mathbf{E}\mathbf{V} + \mathbf{R}\mathbf{V} + Q \quad (21)$$

Det første led repræsenterer ændringen i den totale energi, som består af summen af den kinetiske energi og den interne termiske energi. Det andet led beskriver energifluxen og indeholder bidrag fra konvektiv energitransport, viskøse spændinger og varmeledning. Højresiden repræsenterer arbejdet udført af ydre kræfter samt varmeproduktion og energiudveksling forårsaget af kollisioner.

Herved udleder Braginskii sine tre ligninger for tæthed, impuls og energi udvikling for plasma [7].

Braginskii's kontinuitetsligning

$$\frac{\partial n}{\partial t} + \nabla \cdot (n\mathbf{V}) = 0 \quad (22)$$

Braginskii's momentumligning

$$mn \frac{dV_a}{dt} = -\frac{\partial p}{\partial x_a} - \frac{\partial \pi_{a\beta}}{\partial x_\beta} + en \left(E_a + \frac{1}{c} [\mathbf{V}\mathbf{B}]_a \right) + R_a \quad (23)$$

Braginskii's energitransport ligning

$$\frac{\partial}{\partial t} \left(\frac{nm}{2} V^2 + \frac{3}{2} nT \right) + \frac{\partial}{\partial x_\beta} \left(\left(\frac{mn}{2} V^2 + \frac{5}{2} nT \right) V_\beta + (\pi_{\alpha\beta} V_\alpha) + q_\beta \right) = en\mathbf{E}\mathbf{V} + \mathbf{R}\mathbf{V} + Q \quad (24)$$

Ved modellering af turbulens i randplasmaer er Braginskii's ligninger imidlertid ofte for komplekse til direkte numerisk anvendelse. Derfor introduceres den reducerede drift fluid HESEL modeller, hvor en række fysiske antagelser anvendes til at forenkle beskrivelsen af plasmaets dynamik.

2.2.2 Hesel modellen

I dette projekt anvendes HESEL modellen, som er en n dimensionel drift fluid model udviklet til at beskrive turbulens og transport i tokamak plasmaers rand og SOL regioner. I dette projekt simuleres den i 2 dimensioner. Modellen bygger på Braginskii's fluidbeskrivelse og fremkommer gennem en række approksimationer og reduktioner af de oprindelige ligninger. En detaljeret udledning ligger uden for dette projekts omfang og kan findes i litteraturen [4]. Fokus i dette projekt er derfor ikke på den fulde udledning af HESEL modellen, men på anvendelsen af modellen som fysisk grundlag for de efterfølgende numeriske og maskinlæringsbaserede metoder.

Notationen i Hesel modellen kommer til at følge tæt notationen i [3] og [2]:

$$d_t n + n\mathcal{K}(\phi) - \mathcal{K}(p_e) = \Lambda_n \quad (25)$$

$$d_t^0 \omega^* + \{\phi, p_i\} - \mathcal{K}(p_e + p_i) = \Lambda_{\omega^*} \quad (26)$$

$$\frac{3}{2}d_t p_e + \frac{5}{2}p_e \mathcal{K}(\phi) - \frac{5}{2}\mathcal{K}\left(\frac{p_e^2}{n}\right) = \Lambda_{p_e} \quad (27)$$

$$\frac{3}{2}d_t p_i + \frac{5}{2}p_i \mathcal{K}(\phi) + \frac{5}{2}\mathcal{K}\left(\frac{p_i^2}{n}\right) - p_i \mathcal{K}(p_e + p_i) = \Lambda_{p_i} \quad (28)$$

Det der gør HESEL modellen numerisk effektiv og samtidig fysisk realistisk er, at de tværgående komponenter af momentumligningen reduceres til velkendte plasma drifter. Særligt beskrives plasmaets bevægelse i det tværgående plan ved $E \times B$ driften, mens inertielle effekter indgår gennem den modificerede vorticitet. Derved undgås en direkte løsning af den fulde momentumligning, hvilket reducerer den beregningsmæssige kompleksitet betydeligt.

Den første ligning (25) beskriver bevarelsen af plasmaets partikeltæthed. Ledet $n\mathcal{K}(\phi)$ repræsenterer transport som følge af elektriske felter i et krumt magnetfelt, mens $\mathcal{K}(p_e)$ beskriver drivningen fra elektronstryksgradier. Ligningen udgør dermed grundlaget for beskrivelsen af partikeltransport og dannelsen af tæthedsfluktuationer i HESEL modellen.

Vorticitetsligningen (26) beskriver udviklingen af plasmaets tværgående rotationsbevægelse. Ligningen fremkommer i drift fluid formuleringen ved at kombinere ion og elektronkontinuitetsligningerne under antagelsen om kvasi neutralitet. Den modificerede vorticitet ω^* indeholder både $E \times B$ vorticitet og bidrag fra iontrykket.

Poisson klammen $\{\phi, p_i\}$ beskriver vekselvirkningen mellem det elektrostatiske potentiale og iontrykket, mens krumningsleddet $\mathcal{K}(p_e + p_i)$ repræsenterer drivningen af turbulens gennem trykgradier i det krumme magnetfelt. Højresiden indeholder viskøse og kollisionsbestemte dissipative mekanismer. Ligningen er central for beskrivelsen af turbulens og filamentdynamik i HESEL.

Elektron (27) og iontryk (28) ligningerne beskriver udviklingen af plasmaets termiske energi gennem henholdsvis elektrontrykket p_e og iontrykket p_i . Ligningerne er afledt fra Braginskiis energitransportligninger og indeholder transport med plasmaflowet, kompression i det krumme magnetfelt samt energitransport drevet af trykgradier. Samtidig beskriver de energiudveksling mellem plasmaets termiske energi og kollektive bevægelse.

Lambda termene er fra [3] og her er de givet ved.

$$\Lambda_n = D_e \nabla_{\perp}^2 n - \alpha \left(\tilde{T}_e + \frac{\bar{T}_e}{\bar{n}} \tilde{n} - \tilde{\phi} \right) - \frac{n}{\tau} - \frac{n - n_p}{\tau_p} \quad (29)$$

$$\Lambda_{\omega}^* = D_i \nabla_{\perp}^2 \omega^* - \alpha \left(\tilde{T}_e + \frac{\bar{T}_e}{\bar{n}} \tilde{n} - \tilde{\phi} \right) - \frac{\omega^*}{\tau} + \frac{\rho_s}{L_c} \left[1 - \exp \left(\phi_m - \frac{\phi_s}{T_{e,s}} \right) \right] \quad (30)$$

$$\begin{aligned} \Lambda_{p_e} = & \frac{5}{2} D_e \nabla_{\perp}^2 p_e - \alpha \bar{T}_e \left(\tilde{T}_e + \frac{\bar{T}_e}{\bar{n}} \tilde{n} - \tilde{\phi} \right) - \frac{9p_e}{2\tau} - \frac{p_e - p_{e,p}}{\tau_p} \\ & + \left(\frac{16}{6} - \frac{5}{2} \right) \nabla \cdot (n \nabla_{\perp} T_e) - \frac{T_e}{\tau_{SH}} - \Theta \end{aligned} \quad (31)$$

$$\Lambda_{p_i} = D_i \nabla_{\perp}^2 p_i - D_i T_i \nabla_{\perp}^2 n - \frac{9p_i}{2\tau} + \Theta - \frac{p_i - p_{i,p}}{\tau_p} + p_i \Lambda_{\omega^*} \quad (32)$$

Λ koefficienterne står for at beskrive kollisionsprocesserne, varmeledning, viskøse effekter og parallelle tab og virker dissipativt på systemet. Her repræsenterer D_e og D_i de neoklassiske tværgående transportkoefficienter, som beskriver diffusiv udjævning af henholdsvis partikel, ion, elektron og vorticitetsprofiler. Koefficienten α parametriserer koblingen til parallel driftbølge dynamik på lukkede feltlinjer og driver transport gennem forskelle mellem tæthed, temperatur og potentiale. τ og τ_p beskriver hvor hurtigt feltet tvinges mod profilen, funktionerne med subskript p , beskriver parallelle tab: τ er den karakteristiske tidsskala for tab til (SOL) og sheath, mens τ_p angiver den tidsskala, hvormed plasmaet relaxeres mod de påtvungne kildeprofiler i randregionen. For dette eksperiment er profilerne givet ved:

Funktionerne i HESEL modellen er givet ved:

$$\begin{aligned} n &: n(x, y, t) \\ p_{e,i} &: p_{e,i}(x, y, t) \\ T_{e,i} &: T_{e,i}(x, y, t) \\ \phi &: \phi(x, y, t) \\ \omega^* &: \omega^*(x, y, t) = \nabla^2 \phi + \nabla^2 p_i \end{aligned}$$

Her beskriver n plasmaets partikel tæthed, $p_{e,i}$ er henholdsvis elektron og iontryk, $T_{e,i}$ er elektron og ion temperatur, ϕ er det elektriske potentiale og ω^* er den modificerede vorticitet. Sammenhængen mellem tæthed, tryk og temperatur, er givet ved $p_{e,i} = nT_{e,i}$, hvilket er følge af Boltzmanns konstanten og temperatur samles til et led

$$pV = N\kappa_b T, \quad pV = NT$$

da N er antallet af partikler kan tætheden n udtrykkes på følgende form:

$$p = \frac{N}{V} T, \quad p = nT$$

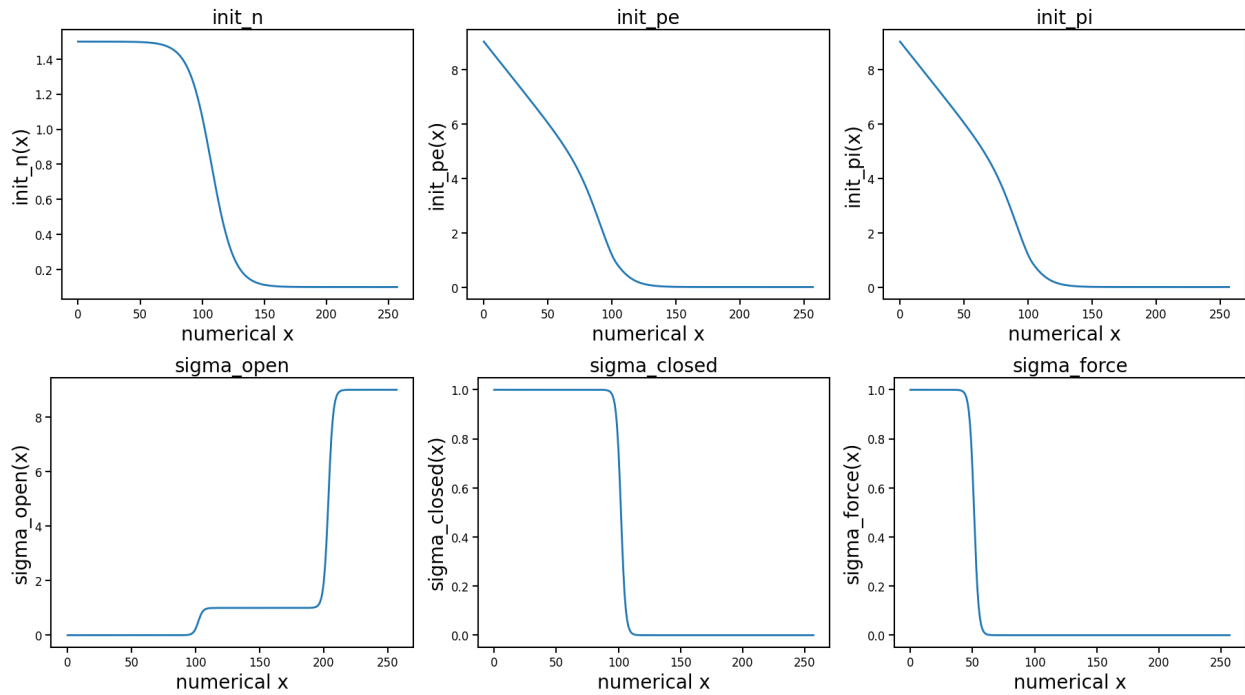


Figure 1: Påtvungne profiler i numerisk domæne, med ghost cells, "init" er n_p , $p_{e,p}$ og $p_{i,p}$, med sigma profiler der bestemmer hvor Λ termene håndhæves. x-aksen har Bohm normaliseret længde: $x = 140/\rho_s$

Når HESEL modellen simuleres, omskrives ligningerne til Bohm normaliserede enheder, hvilket reducerer antallet af fysiske parametre, forbedrer den numeriske stabilitet og gør det muligt at beskrive plasmaets dynamik i en dimensionsløs form uafhængigt af de valgte referenceværdier. Bohm normalisering betyder at visse variabler ganges med konstanter for at gøre dem dimensionsløse.

$$t \rightarrow \omega_{ci,0}t, \quad x \rightarrow \frac{x}{\rho_s}, \quad n \rightarrow \frac{n}{n_0}, \quad T_{e,i} \rightarrow \frac{T_{e,i}}{T_{e0}}, \quad p_{e,i} \rightarrow \frac{p_{e,i}}{n_0 T_{e0}},$$

Det bemærkes at i literaturen bruges (x, y, z) koordinat indeksering, men når Hesel simuleres bruges (x, z, y) indeksering. I metode afsnittet vil (x, y, z) koordinat indeksering bruges men i 3 bruges (x, z, y) indeksering. Dette ændre ikke på noget fysisk eller matematisk men blot notationen.

Til sidst er det nyttigt at introducere den række operatorer, som anvendes i HESEL

modellen for at gøre ligningerne mere kompakte og læsbare.

$$\mathcal{K}(f) = -\frac{\rho_s}{R} \partial_y f \quad (33)$$

$$\{f, g\} = \partial_x f \partial_y g - \partial_y f \partial_x g \quad (34)$$

$$\mathrm{d}_t^0 f = \partial_t f + \{\phi, f\} \quad (35)$$

$$\mathrm{d}_t f = \partial_t f + \frac{1}{B} \{\phi, f\} \quad (36)$$

$$\bar{f} = \frac{1}{L_y} \int_0^{L_y} f \mathrm{d}y \quad (37)$$

$$\tilde{f} = f - \bar{f} \quad (38)$$

$$\nabla = \begin{pmatrix} \partial_x \\ \partial_y \\ \partial_z \end{pmatrix} \quad (39)$$

$$\nabla^2 = \partial_x^2 + \partial_y^2 + \partial_z^2 \quad (40)$$

$$\nabla_{\perp} = \begin{pmatrix} \partial_x \\ \partial_y \end{pmatrix} \quad (41)$$

$$\nabla_{\perp}^2 = \partial_x^2 + \partial_y^2 \quad (42)$$

$$(43)$$

De sidste parametre er alle skalarer der bestemmer de geometriske, numeriske og fysiske størrelser. Disse kan læses om i [4]

2.3 Numeriske metoder

Simuleringsmotoren er bygget til numerisk at løse koblede partielle differentiaalligninger. I dette projekt anvendes et todimensionalt domæne, så PDE systemet kan skrives som

$$\frac{\partial \mathbf{u}}{\partial t} = \mathcal{F}(t, x, y, \mathbf{u}, \nabla \mathbf{u}, \nabla^2 \mathbf{u}), \quad (x, y) \in \Omega, \quad (44)$$

hvor $\mathbf{u}(t, x, y) \in \mathbb{R}^m$ repræsenterer ét felt eller et koblet system af m felter. Grundideen er at diskretisere rummet først og derefter udvikle den diskrete løsning i tid. Denne fremgangsmåde kaldes metode af linjer.

2.3.1 Rumlig diskretisering

Domænet vælges som et rektangel

$$\Omega = [x_{\min}, x_{\max}) \times [y_{\min}, y_{\max}),$$

og opdeles i et uniformt gitter med N_x punkter i x retningen og N_y punkter i y retningen. Gitterpunkterne kan skrives som

$$x_i = x_{\min} + i\Delta x, \quad y_j = y_{\min} + j\Delta y,$$

hvor

$$\Delta x = \frac{x_{\max} - x_{\min}}{N_x}, \quad \Delta y = \frac{y_{\max} - y_{\min}}{N_y}.$$

Den diskrete løsning betegnes ved $\mathbf{u}_{i,j}(t)$, som approksimerer den kontinuerte løsning i punktet (x_i, y_j) . For koblede systemer er $\mathbf{u}_{i,j}(t)$ en vektor. Når ghost celler indføres, lader man de fysiske gitterpunkter have indeks $i = 1, \dots, N_x$ og $j = 1, \dots, N_y$, mens de ekstra randceller får indeks 0 og $N_x + 1$ i x retningen samt 0 og $N_y + 1$ i y retningen.

Gitteret udvides med ghost celler omkring randen, så de samme stencilformler kan anvendes i hele det indre domæne. For periodiske randbetingelser kopieres værdierne fra den modsatte side, så der diskret gælder

$$\mathbf{u}_{0,j} = \mathbf{u}_{N_x,j}, \quad \mathbf{u}_{N_x+1,j} = \mathbf{u}_{1,j},$$

og tilsvarende i y retningen.

2.3.2 Numeriske differentialoperatorer

De rumlige afledte approksimeres med centrale differenser af anden orden. For et skalarfelt u fås

$$(\partial_x u)_{i,j} \approx \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x}, \quad (\partial_y u)_{i,j} \approx \frac{u_{i,j+1} - u_{i,j-1}}{2\Delta y}. \quad (45)$$

Tilsvarende approksimeres de andenordens afledte ved

$$(\partial_{xx}u)_{i,j} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}, \quad (\partial_{yy}u)_{i,j} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}. \quad (46)$$

Heraf følger den diskrete Laplace operator

$$(\nabla^2 u)_{i,j} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}.$$

Hvis PDE systemet indeholder advektionsled, skrives disse diskret som

$$\mathbf{v} \cdot \nabla u \approx v_x (\partial_x u)_{i,j} + v_y (\partial_y u)_{i,j}.$$

Denne diskretisering har truncationsfejl af størrelsesorden $\mathcal{O}(\Delta x^2 + \Delta y^2)$ for glatte løsninger.

Når alle rumlige operatorer er diskretiseret, kan PDE systemet omskrives til et stort system af ordinære differentialligninger,

$$\frac{d\mathbf{U}}{dt} = \mathbf{F}(t, \mathbf{U}), \quad (47)$$

hvor $\mathbf{U}(t)$ samler alle gitterværdier for alle felter.

2.3.3 Tidsintegration

Til tidsudviklingen anvendes den klassiske Runge–Kutta metode af fjerde orden, RK4. Givet den semidiskrete tilstand $\mathbf{U}^n \approx \mathbf{U}(t_n)$ og et tidsstep Δt_n beregnes

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{F}(t_n, \mathbf{U}^n), \\ \mathbf{k}_2 &= \mathbf{F}\left(t_n + \frac{\Delta t_n}{2}, \mathbf{U}^n + \frac{\Delta t_n}{2} \mathbf{k}_1\right), \\ \mathbf{k}_3 &= \mathbf{F}\left(t_n + \frac{\Delta t_n}{2}, \mathbf{U}^n + \frac{\Delta t_n}{2} \mathbf{k}_2\right), \\ \mathbf{k}_4 &= \mathbf{F}(t_n + \Delta t_n, \mathbf{U}^n + \Delta t_n \mathbf{k}_3), \end{aligned}$$

hvorefter løsningen opdateres ved

$$\mathbf{U}^{n+1} = \mathbf{U}^n + \frac{\Delta t_n}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (48)$$

RK4 er en eksplicit metode med høj nøjagtighed, men stabiliteten afhænger stadig af de rumlige skalaer og styrken af de fysiske led.

Den samlede numeriske strategi er derfor et uniformt gitter med periodiske randbetingelser, centrale differensoperatorer i rummet, metode af linjer til at danne et stort ODE system og RK4 til tidsudviklingen.

2.4 Tilgang til eksperiment

Formålet med eksperimenterne er at undersøge, i hvilken grad fysikinformede neurale netværk kan lære og reproducere dynamikken i plasmaets randregion, som beskrevet af HESEL modellen. Mere specifikt ønskes det at undersøge, hvor præcist en model kan approksimere udviklingen af systemets tilstand fra ét tidssteg til det næste. Givet en tilstand $u(t)$ er målet således at lære en approksimation af tidsudviklingsoperatoren

$$u(t + \Delta t) \approx \mathcal{F}_\theta(u(t)),$$

hvor $\mathcal{F}_\theta$ repræsenterer den lærte model med parametre θ . Hvis en sådan approksimation kan opnås med høj nøjagtighed, vil modellen potentielt kunne fungere som en surrogatmodel for dele af den numeriske løsning og dermed reducere behovet for kostbare beregninger.

HESEL modellen beskriver et stærkt ikke lineært og turbulent system, hvor små variationer i begyndelsesbetingelserne kan føre til markant forskellige udviklingsforløb. Som følge heraf er det relevant først at undersøge modellens evne til at rekonstruere simuleringer genereret fra kendte begyndelsesbetingelser. Dette giver et mål for, hvor godt modellen er i stand til at lære den underliggende dynamik i de data, den er trænet på.

Et centralt spørgsmål er ligeledes modellens evne til at generalisere til tidligere usete begyndelsesbetingelser. Hvis modellen kan forudsige udviklingen af simuleringer, som ikke indgår i træningsdatasættet, indikerer det, at den har lært generelle strukturer i dynamikken frem for blot at memorisere specifikke løsninger. En sådan egenskab vil være særligt værdifuld i forbindelse med hurtig udforskning af nye parameterrum og scenarier, hvor numeriske simuleringer endnu ikke er udført.

Derudover undersøges modellens langsigtede stabilitet gennem autoregressiv tidsintegration. Her anvendes modellens egen prædiktion som input til næste tidssteg,

$$u_{n+1} = \mathcal{F}_\theta(u_n),$$

således at fejl kan akkumulere over tid. Formålet er at undersøge, hvor langt frem i tiden modellen kan forudsige systemets udvikling, før løsningen afviger væsentligt fra den numeriske reference. Dette giver et mål for modellens evne til at opretholde fysisk meningsfulde løsninger uden løbende korrektion fra simuleringsdata.

Endelig undersøges modellens evne til at generalisere på tværs af rumlige opløsninger. Målet er at afklare, om en model trænet på et grovere gitter kan anvendes til at approksimere løsninger på finere gitter. En sådan skalerbarhed er særligt interessant i plasmafysiske anvendelser, hvor højopløselige simuleringer ofte udgør en væsentlig beregningsmæssig omkostning. Hvis modellen kan overføre information mellem forskellige opløsninger, kan dette potentielt reducere den samlede beregningstid og gøre det muligt at generere tilnærmede højopløselige løsninger betydeligt hurtigere end ved traditionelle numeriske metoder.

2.4.1 Opstilling af eksperimenter

Til at undersøge disse spørgsmål anvendes et sæt af numeriske simuleringer genereret med BOUT++ og BOUT HESEL. Det blev valgt at simulere nogle begyndelsesbetingelser,

der var hurtigt blev kaotiske og svære at simulere. Dette blev valgt for at udfordre modellen og undersøge, hvor godt den kunne lære og generalisere i et komplekst dynamisk regime. Disse begyndelsesbetingelser kaldes standarden.

Der blev genereret 6 forskellige simuleringer. Tre af disse simuleringer var standarden, hvor det kun var det seed der blev ændret. De resterende tre simuleringer var også standard hvor kun en parameter blev ændret. Disse var tidssteg afstanden, gitteropløsningen og den fysiske parameter.

2.4.2 Valg af model arkitektur

Herefter blev der udført design af eksperimenter på to af standard simuleringerne for at bestemme netværksarkitektur. Som udgangspunkt vælges et almindeligt fuldt forbundet neuralt netværk. Dette er i tråd med både den klassiske formulering for neurale netværk til differentialligninger i [5] og den senere PINN formulering i [6], hvor løsningen repræsenteres med simple feedforward netværk. Raissi et al. viser desuden, at glatte aktiveringsfunktioner som tanh og en vis netværksdybde giver en mærkbar forbedring i approksimationskvaliteten. På den baggrund vælges her et simpelt, men forholdsvis bredt søgeområde bestående af 2, 4, 6 eller 8 skjulte lag og hidden dimensioner fra 256 til 4096. For aktiveringsfunktionerne undersøges både tanh, og SiLU som er velafprøvet i PINN litteraturen. Der undersøges

Her efter udføres der design af eksperimenter for forskellige netværksarkitekturer før den endelige arkitektur vælges.

Der bliver undersøgt 2 forskellige input og output formater størrelser. Begge får de to forrige tidssteg som input, samt deres koordinater de skal forudsige. Den første model får blot ét punkt og de omkringliggende punkter som input, og skal forudsige det næste tidssteg for det samme punkt. Den næste model får hele x akse og de to nabocellerne som input, og skal forudsige det næste tidssteg for hele x akse. Den første model der kun skal forudsige ét punkt ad gangen, bliver softconstrained i mens modellen der skal forudsige hele x akse bliver hardconstrained. Dette valg er baseret på, at den første model kan fokusere på at lære de lokale detaljer i løsningen, mens den anden model kan lære de globale strukturer i løsningen. Grunden til ikke at modellen skal have hele billedet som input og skal forudsige hele billedet som output er grundet at neurale netværk forudsiger gennemsnitlige værdier, og derfor ikke er i stand til at forudsige de små detaljer i løsningen, som er vigtige for at opnå en god løsning. Derfor skal modellen have et mindre input og output format, så den kan fokusere på de vigtigste detaljer i løsningen.

3 Implementering

3.1 Overordnet struktur

Her gennemgås hele git repositoriet for dette eksperiment [8].

3.1.1 Pipeline for modellen

Projektets eksperimentelle del er bygget op som en sammenhængende pipeline fra numerisk plasmasimulation til træning af SciML modeller.

Udgangspunktet er output fra BOUT++/Hesel simuleringer. Hver simulering består dels af en konfigurationsfil, `BOUT.settings`, dels af et sæt filer af typen `BOUT.dmp.*.nc` med de beregnede felter. Disse data bliver først behandlet i mappen `hesel_scraper`, som fungerer som bindeled mellem råsimulation og de efterfølgende SciML modeller.

De to centrale filer i denne del er `bout_dump.py` og `bout_phys.py`. Rollen af de vigtigste moduler kan opsummeres som:

- `hesel_scraper/bout_dump.py`: indlæser dump-filerne, fortolker `BOUT.settings`, samler felter på et fælles gitter og udleder normaliserede fysiske parametre.
- `hesel_scraper/bout_phys.py`: konstruerer de fysiske restriktioner, dvs. initialbetingelser, randbetingelser og residualer for de evolutionære ligninger.
- `hesel_scraper/api/operators.py`: indeholder differentiationsoperatorer baseret på utograd, som bruges til at beregne gradienter og Laplace operatorer direkte på model outputet.
- `hesel_scraper/api/dump_helper.py`: hjælper med sikker parsing af udtryk og funktionsdefinitioner fra `BOUT.settings`, så simulations opsætningen kan genbruges direkte i modellen.

Når simulationen er oversat til en intern repræsentation, overtager mapperne i SciML selve læringsproblemet. Strukturen er her bevidst delt i to principielt forskellige modelklasser:

- `SciML/PINN`: den punktvis fysik informerede model, hvor netværket forudsiger feltværdier i et enkelt punkt ud fra lokale historiske data og koordinater. Denne model er formuleret som en *soft constraint* model.
- `SciML/PINN_z` og `SciML/PINO_z`: to linjebaserede modeller, som arbejder på hele snit i x retningen og lokale vinduer i z retningen. Begge er formuleret som *hard constraint* modeller.

Inden for den anden klasse adskiller `PINN_z` og `PINO_z` sig derefter primært ved selve netværkstypen: `PINN_z` bruger et MLP baseret netværk, mens `PINO_z` bruger en operator-baseret FNO lignende arkitektur. Den vigtigste strukturelle forskel i dette afsnit er derfor

ikke mellem PINN_z og PINO_z, men mellem den punktwise soft constraint model og de to linjebaserede hard constraint modeller.

Selvom modelretningerne er implementeret forskelligt, bygger de alle på samme grundlæggende objekt fra `hesel_scraper`: en konsistent beskrivelse af data, koordinater, normalisering og fysisk residual. Det er derfor naturligt at se `hesel_scraper` som det fysiske og datamæssige fundament, mens `SciML` mapperne specificerer selve læringsarkitekturen.

3.1.2 Sammenhæng mellem simulation, data og model

Matematisk kan Hesel simuleringen betragtes som en diskret approksimation af et kontinuert dynamisk system på domænet

$$\Omega = [0, L_x] \times [0, L_z] \times [0, T].$$

På dette domæne beskrives plasmaets tilstand ved de fire primære felter

$$\mathbf{u}(x, z, t) = \begin{bmatrix} \ln n(x, z, t) \\ \ln p_e(x, z, t) \\ \ln p_i(x, z, t) \\ \phi(x, z, t) \end{bmatrix},$$

hvor n er densitet, p_e og p_i er elektron og iontryk, og ϕ er det elektrostatiske potentiale. Logaritmerne bruges for at gøre simuleringen numerisk stabil og undgå negative værdier for n , p_e og p_i .

Ud fra disse felter dannes også afledte størrelser. I implementeringen beregnes blandt andet vorticiteten som $\omega^*(x, y, t) = \nabla^2 \phi + \nabla^2 p_i$, som var givet i 2.2. Da vorticiteten kan udledes direkte fra ϕ og p_i , er den ikke inkluderet som et separat felt i modellen, men udregnes i `hesel_scraper/bout_phys.py`.

Fra simulationen fås et diskret gitter (x_i, z_j, t_n) , og i `bout_dump.py` organiseres data derfor som tensorfelter af typen

$$\mathbf{u}_{n,i,j} = \mathbf{u}(t_n, x_i, z_j).$$

Samtidig udledes de normaliserede parametre, der indgår i Hesel ligningerne, eksempelvis transportkoefficienter, karakteristiske tider og længdeskalaer. Dermed bliver simulationens numeriske opsætning oversat til et model klart objekt, som både indeholder rådata og den fysik, der skal bruges til træning.

Den punktwise PINN behandles som sin egen modeltype. Her konstrueres hvert træningseksempel som en lokal afbildning

$$(\mathbf{s}_k, \mathbf{c}_k) \mapsto \mathbf{y}_k,$$

hvor $\mathbf{c}_k = (t_n, x_i, z_j)$ er den fysiske koordinat, $\mathbf{y}_k = \mathbf{u}_{n,i,j}$ er mål vektoren, og $\mathbf{s}_k$ er et lokalt state vindue bestående af de to foregående tidsskridt og det 3×3 vindue omkring disse

punkter. Modellen kan derfor ideelt skrives som

$$\hat{\mathbf{u}}_\theta(t_n, x_i, z_j) = F_\theta(t_n, x_i, z_j, \mathbf{s}_k),$$

hvor θ er netværkets parametre.

Den punktvis PINN er en soft constraint formulering, fordi de fysiske krav ikke bygges direkte ind i selve modelansatsen, men i stedet håndhæves gennem tabsfunktionen. Hvis modellen skrives som $\hat{\mathbf{u}}_\theta$, bliver initialbetingelser, randbetingelser og ligninger derfor kun opfyldt approksimativt gennem residualled som straffes under træningen. Det betyder også, at denne model naturligt beskrives som en punktvis regressionsmodel med fysisk regularisering.

De to modeller PINN_z og PINO_z behandles derimod samlet som en anden modeltype. Her generaliseres ideen fra enkeltpunkter til hele snit i x akse. Input er derfor ikke kun lokale punkter, men linjer, der bevarer mere af den rumlige struktur. Det betyder, at modellerne i højere grad lærer en operator mellem felter frem for blot en lokal regressionsregel. Den præcise repræsentation af input og output beskrives senere i kapitlet, men overordnet er formålet det samme: at approksimere tidsudviklingen af Hesel felterne.

For PINN_z og PINO_z bruges der en hard constraint konstruktion. Her skrives modeloutputtet ikke blot som et frit netværksoutput, men indlejres i en ansats af formen

$$\hat{\mathbf{u}}_\theta = \mathbf{g} + \sigma(t)(\mathbf{H} - \mathbf{g} + \mathbf{D}\mathbf{N}_\theta),$$

hvor $\mathbf{g}$ repræsenterer initialtilstanden, $\mathbf{H}$ er en hjælpefunktion der opfylder randbetingelserne, $\mathbf{D}$ er en dæmpningsfunktion som går mod nul ved randen, og $\sigma(t)$ sørger for korrekt kobling til begyndelsestiden. Dermed bliver dele af initial og randbetingelserne opfyldt konstruktivt for alle θ , i stedet for kun at blive presset ned via ekstra tabsled.

Den centrale sondring i projektet er derfor:

- PINN: punktvis, soft constraint og styret af en tabsfunktion med eksplicitte data, initial, rand og ligningsled.
- PINN_z og PINO_z: linjebaserede, hard constraint og bygget op omkring en ansats, hvor dele af initial og randbetingelserne håndhæves direkte i modeloutputtet.

Det væsentlige SciML element er, at modellerne ikke kun trænes mod data, men også med de underliggende ligninger. I `bout_phys.py` defineres derfor tre typer residualer:

- data residualer, som måler forskellen mellem modelens forudsigelse og simulationsdata,
- rand og initial residualer, som tvinger modellen til at respektere start tilstanden og de fysiske grænser,
- ligningsresidualer, som håndhæver Hesel dynamik.

For de fysiske ligninger bruges en generel struktur af typen

$$\partial_t q = \mathcal{R}(q) + \Lambda(q),$$

hvor $q \in \{\ln n, \ln p_e, \ln p_i, \omega\}$. Her samler $\mathcal{R}(q)$ de egentlige dynamiske transportled, mens $\Lambda(q)$ samler de øvrige bidrag såsom diffusiv transport, parallel dynamik, profilforcering og gulvled. I koden omsættes dette til residualer påformen

$$r_q = \partial_t q - \mathcal{R}(q) - \Lambda(q),$$

og målet er, at $r_q \approx 0$ i hele det relevante træningsdomæne.

Den samlede træning kan derfor beskrives som minimering af en vægtet tabsfunktion

$$\mathcal{L} = \lambda_{\text{da}} \mathcal{L}_{\text{data}} + \lambda_{\text{eq}} \mathcal{L}_{\text{eq}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}},$$

hvor de fire led svarer til henholdsvis datafit, ligningsresidual, initialbetingelser og randbetingelser. Hele eksperimentet kan dermed forstås som en søgen efter modelparametre θ , så den lærte løsning både ligner simulationsdata og opfører sig som en fysisk konsistent løsning til Hesel systemet.

Forskellen mellem modeltyperne ligger i, hvordan disse residualer bruges. For den punktvisse PINN indgår data, initial, rand og ligningsresidualer alle direkte i tabsfunktionen. For PINN_z og PINO_z er initial og randbetingelser i høj grad bygget ind i ansatsen, og derfor trænes disse modeller primært med dataresidual og ligningsresidual.

For den punktvisse soft constraint model er denne tabsfunktion den direkte træningsmålsætning. For de to hard constraint modeller reduceres træningsmålsætningen i praksis til de led, som ikke allerede er håndhævet konstruktivt, hvilket i implementeringen svarer til en kombination af datafit og ligningsresidual.

4 Data og eksperiment

Den overliggende hypotese for de endelige eksperimenter er følgende:

- En diskret PINN kan approksimere næste tids state
- En diskret PINN kan iterativt approksimere tidsudviklingsoperatoren
- En diskret PINN kan approksimere kendte og ukendte fysiske begyndelsestilstande
- En diskret PINN kan approksimere kendte fysiske begyndelsestilstande på forskellige grid opløsninger

4.1 Opsætning og generering af data

4.1.1 BOUT opsætning

BOUT++ og BOUT-HESEL blev opsat på DTU's HPC-cluster i et separat softwaremiljø, hvor de nødvendige moduler for MPI, FFTW, NetCDF og Intel compiler først blev loadet. Dette var nødvendigt, fordi både kompilering og kørsel af simuleringerne afhænger af parallelisering og eksterne numeriske biblioteker. Derudover blev `OMP_NUM_THREADS` sat til 16, så opsætningen matchede den ønskede brug af clusterets CPU-ressourcer.

Selve installationen blev udført i to trin. Først blev BOUT++ klonet fra det officielle repository, sat til version `v4.3.2`, konfigureret med `./configure` og derefter kompileret med `make -j16`. BOUT++ fungerer her som den generelle simuleringsramme, som håndterer numerisk løsning, gitterstruktur og parallel udførelse. Herefter blev BOUT-HESEL klonet med tilhørende submoduler. I denne del af opsætningen blev de nødvendige komponenter fra HESEL-Common koblet til projektet, så HESEL-modellen kunne bygges oven på den allerede kompilerede BOUT++-installation. Til sidst blev BOUT-HESEL også kompileret med `make -j16`.

Når både BOUT++ og BOUT-HESEL var bygget, kunne simuleringerne køres med `mpirun -np 4 ./hesel -d <folder med BOUT.inp fil>`. Her læser programmet sin konfiguration fra den tilhørende `BOUT.inp`-fil, som specificerer blandt andet gitteropløsning, tidsstep, randbetingelser og fysiske parametre. Denne opsætning gjorde det muligt at generere konsistente HESEL-simuleringer på clusteret, som efterfølgende kunne bruges som datagrundlag for de læringsbaserede modeller. Dette projekts opsætning af BOUT++ og BOUT-HESEL er dokumenteret i detaljer i bilag 8.2, hvor alle nødvendige kommandoer og trin er gennemgået for at sikre reproducerbarhed.

4.1.2 Data generering

Til dette er der generet 6 simuleringer. Herfra er 3 standard simuleringer og 3 simuleringer hvor eneste ændring er: tidsteppet, fysikken og grid opløsningen. Alle simuleringskonfigurationsfiler der bestemmer begyndelsestilstande, randbetingelser, grid opløsninger, fysiske parametre og tilfældige seed ligger i folderen `sim_data` i Github repositoryet.

- 3 standard simuleringer starter med `sim_data/Alexander_*` " Her er `*` en plads for forskellige seed"
- Folderen med modificeret tidsteppet starter med `sim_data/Alexander__t_n`
- Folderen med modificeret fysik starter med `sim_data/Alexander__fys_R_1`
- Folderen med modificeret grid opløsning starter med `sim_data/Alexander__grid_c`

Standard input filen `sim_data/Alexander_phi_0/BOUT.inp` er lagt i bilag 8.1. Resten kan findes inde i folderen `sim_data`.

Alle simuleringer blev udført på Danmarks Tekniske Universitet (DTU)'s High Performance Computing (HPC) cluster, med 16 CPU kerner. Hver simulering tog ikke længere end 1 time. Submit filen til simuleringen ligger her: `sim_data/submit_simulering.sh`. Træningen af modellerne blev udført lokalt. Derofr blev filer downloadet fra HPC clusteret til lokal maskine med

```
rsync -avh -progress ssh:<sti til data> <sti til lokal folder>
```

Det er ikke forventet at download skulle ødelegge data'en når den komprimeres og downloades med `rsync`.

For simuleringen med modificeret tidstep sættes variablen `timestep = 25`. Det betyder at modellen skal forudsige længere frem i tiden end hvad den er trænet på.

For simuleringen med modificeret fysik er følgende variabler ændret til

`Z_eff = 1.7`

`Bt = 2.32`

`Te0 = 75.6`

`Ti0 = 75.6`

Disse blev vurderet til at være mest almindelige fysiske parametre at variere. For simuleringen med modificeret grid opløsning blev ændre til at være `nx = 514, nz = 512`

Alle filer, både standard og modificerede, havde forskellige seeds for at sikre at modellerne ikke overfittede på specifikke begyndelsestilstande.

4.1.3 Data visualisering

Selve visualiseringen af data er udført i `hesel_scraper/visulizer.ipynb`. Her bruges klassen `BOUTHESELInfo` fra `bout_dump` biblioteket. Denne klasse læser data'en via `xbout` og gemmer den som tensorer så python nemmere kan arbejde med det.

5 Resultater

Her kommer der til at være en masse om resultaterne

6 Konklusion og diskussion

Her står der en masse om mine tanker til forbedringer og videre arbejde. Det er ikke sikkert at det

7 Fremtidigt arbejde

List of Figures

- 1 Påtvungne profiler i numerisk domæne, med ghost cells, "init" er n_p , $p_{e,p}$ og $p_{i,p}$, med sigma profiler der bestemmer hvor Λ termene håndhæves. x-aksen har Bohm normaliseret længde: $x = 140/\rho_s$ 13

List of Tables

Nomenclature

BTE Boltzman transport ligning

HESEL Hot Edge SOL Electrostatic

SciML Scientific Machine Learning

SOL Scrape off layer

References

- [1] ITER Organization, “The way to new energy,” 2019. Accessed: 2026-05-11.
- [2] A. S. Thrysøe, M. Løiten, J. Madsen, V. Naulin, A. H. Nielsen, and J. J. Rasmussen, “Plasma particle sources due to interactions with neutrals in a turbulent scrape-off layer of a toroidally confined plasma,” *Physics of Plasmas*, vol. 25, p. 032307, 03 2018.
- [3] A. Nielsen, J. Rasmussen, J. Madsen, G. Xu, V. Naulin, J. Olsen, M. Magnussen, S. Hansen, N. Yan, L. Tophøj, and B. Wan, “Numerical simulations of blobs with ion dynamics,” *Plasma Physics and Controlled Fusion*, vol. 59, no. 2, 2017.
- [4] A. Thrysøe, M. Løiten, J. Madsen, V. Naulin, A. Nielsen, and J. Rasmussen, “Plasma particle sources due to interactions with neutrals in a turbulent scrape-off layer of a toroidally confined plasma,” *Physics of Plasmas*, vol. 25, no. 3, 2018.
- [5] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [6] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations,” *arXiv preprint arXiv:1711.10561*, 2017.
- [7] S. I. Braginskii, “Transport processes in a plasma,” *Reviews of Plasma Physics*, 1, 205., vol. 1, pp. 205–311, 1965. Accessed: 2026-06-11.
- [8] A. Antoft, “Bachelorprojekt: Physics informed neural networks and operators for hesel simulations,” 2026. GitHub repository, accessed: 2026-06-17.

8 Appendix A

8.1 Input fil for konfiguration af simulering

Dette er standard konfigurationsfil for simuleringer af BOUT-HESEL.

Listing 1: BOUT.inp for datasættet Alexander_phi_0

```

1  #=====
2  #=====
3  t_end = timestep*nout # total simulation time
4  timestep = 15 # time step length of outputted data
5  nout = 1000 # number of outputted time steps
6
7  myg = 0 # no need for Y communications, so no guard cells
8  mxg = 1 # guard cell number in the boundary of local CPU
9  #grid = alec_data/tcv/64465/experimental_data/grid.nc
10 #=====
11 #=====
12 [mesh]
13 nx = 256+2*mxg # internal grid points plus guard cells along x-axis
14 ny = 1 # y is along magnetic field, set to 1
15 nz = 256 # internal grid points along z-axis
16
17 Lx = 140 # simulation length along x-axis
18 Lz = 140 # simulation length along z-axis
19
20 dx = Lx/(nx-1-2*mxg) # grid spacing along x-axis
21 dy = 1.0 # grid spacing along y-axis
22 dz = Lz/(nz-1) # grid spacing along z-axis
23
24 xl = x*(Lx+2*mxg*dx) # grid points coordinates along x-axis
25 zl = z*Lz # grid points coordinates along z-axis
26 #=====
27 #=====
28 [mesh:ddx] # methods used for x derivative terms
29 first = C2 # d/dx (f)
30 second = C2 # d^2/dx^2 (f)
31 upwind = W3 # v_x d/dx (f)
32 flux = U1 # d/dx (v_x f)
33
34 [mesh:ddy] # methods used for y derivative terms
35 first = C2 # d/dx (f)
36 second = C2 # d^2/dx^2 (f)
37 upwind = W3 # v_x d/dx (f)
38 flux = U1 # d/dx (v_x f)
39

```

```

40 [mesh:ddz] # methods used for z derivative terms
41 first = C2 # d/dx (f)
42 second = C2 # d^2/dx^2 (f)
43 upwind = W3 # v_x d/dx (f)
44 flux = U1 # d/dx (v_x f)
45 #=====
46 #=====
47 [laplace] # boundary condition for phi
48 inner_boundary_flags = 0
49 outer_boundary_flags = 3
50
51 [solver] # ddt solver settings
52 type = pvide # solver type for solving ddt
53 atol = 1.0e-10 # absolute tolerance
54 rtol = 1.0e-5 # relative tolerance
55 timestep = 1. # Suggested init timestep for numerical methods
56 mxstep = 1000000 # max steps before result is deemed not to converge
57 #=====
58 #=====
59 [hesel]
60 right_handed_coord = false # switch for converting BOUT++ left-handed coordinate
    to HESEL right-handed coordinate
61 interchange_dynamics = true # switch for interchange dynamics, left hand side
    parts of HESEL equations
62 parallel_dynamics = true # switch for parallel dynamics
63 perpendicular_dynamics = true # switch for perpendicular dynamics
64 plasma_neutral_interactions = 0 # switch for plasma neutral interactions 15 - all
    on, 16 dynamic non interacting
65 kinetic_plasma_neutral_interactions = 1 # switch for kinetic plasma neutral
    interactions
66 force_profiles = true # switch for forcing profiles
67 floor_profiles = false # switch for floor profiles
68 static_plasma = 0 # bitwise switch for making plasma static, (n, pe, pi, vort, te,
    ti) <-> (1, 2, 4, 8, 16, 32)
69
70
71 Z_eff = 1.8
72
73 Bt = 2.28 # toroidal magnetic field at magnetic axis
74 q = 4.8 # safety factor (95%)
75 Te0 = 78.6 # reference electron temperature
76 Ti0 = 78.6 # reference ion temperature
77 n0 = 3.4e+19 # reference electron density
78 lconn = 40 # connection length
79 Rminor = 1 # minor radius
80 Rmajor = 3 # major radius

```

```

81 lblob = 2*q*Rmajor # ballooning length, -1 use inner definition: lblob = q*R
82 A = 2 # ion mass number. m_i = A*m_p
83 Z = 1 # ion charge
84 Mach = 0.5 # Mach number
85
86
87 e = 1.60e-19
88 mp = 1.67262158e-27
89 mi = A*mp
90 B0 = Bt*Rmajor/(Rmajor+Rminor)
91
92 oci = e*Z*B0/mi
93 cs = sqrt(e*Te0/mi)
94 rhos = cs/oci
95
96 # parallel dynamics
97 parallel_sheath_damping = 3 # 0 = off, 1 = cs0+average field, 2 = average field, 3
    = full field
98 parallel_advection_damping = 3 # 0 = off, 1 = cs0, 2 = avgcsHot, 3 = csHot
99 parallel_conduction = 1 # 0 = off, 1 = electron, 2 = electron + ion
100 parallel_drift_wave = 2 # 0 = off, 1 = Te0, 2 = avg_te, 3 = te How tau dw depends
    on te
101 reciprocal_approx = 3 # 1: constant fields, 2: avg fields, 3: no approximation
102
103 # perpendicular dynamics
104 collisional_model = 2 # switch for neo-classical collision model on RHS; 0: off,
    1: simple, 2: standard, 3: full
105 perpend_heat_exchange = true
106 perpend_viscous_heating = true
107 ti_over_te = 3 # approximation of ti/te, 1: Ti0/Te0, 2: ZAvg(ti/te), 3: no
    approximation
108 diffusion_coeff = 1 # approximation for diffusion coefficients, 1: n0+te0 (std),
    2: avg_n+avg_te, 3: n+te, 4: avg_n+te (full), 5: n+avg_te, 6 n0+avg_te
109 neoclass_correction_factor = -1 # neo-classical correction of diffusion
    coefficients
110 qdelta_approx = 3 # approximation of nuei in Qdelta, 0: off, 1: nuei0, 2: average
    field, 3: full field
111
112 # bracket options
113 bracket_method = 2 # 0 = std, 1 = simple, 2 = arakawa
114
115 # force profiles options
116 force_time = 50 # force profiles time
117
118 # floor profiles options
119 floor_n = 0.005

```

```

120 floor_pe = 0.000025
121 floor_pi = 0.000025
122 floor_time = 50 # floor profiles time
123
124 # variables used for profile generation
125 x_lcfs = 0.40 # x coordinate for LCFS, x range is [0, 1]
126 x_wall = 0.80 # x coordinate for WALL, x range is [0, 1]
127 d_lcfs = 2 # step width of smoothing function between inner and SOL regions
128 d_wall = 2 # step width of smoothing function between wall and SOL regions
129 d_force = 2 # step width of smoothing function for forcing profiles
130 wall_amp = 8 # parallel damping in wall region is given as a factor wall_amp times
    the damping in the SOL region
131 edge_width = x_lcfs*mesh:Lx # width of edge region
132 sol_width = (x_wall-x_lcfs)*mesh:Lx # width of SOL region
133 force_width = 0.5 * edge_width # width of force region
134 xr = mesh:x1 - edge_width # x coordinate with LCFS is origin
135 #=====
136 #Include neutrals
137 #=====
138 [neutrals]
139
140 atomicIonization = true # H + e -> H+ + 2e
141 molecularIonization = true # H2 + e -> H2+ + 2e -> H + H+ + 2e
142 chargeExchange = true # H + H+ -> H+ + H
143 dissociation = true # H2 + e -> 2H + e
144 elasticCollisions = false
145
146 neutralDynamics = 0 # solve for neutral dynamics
147
148 nColdFlux = -1.5e21 # m^-2s^-1
149
150 local_diffusion = false
151
152 Tcold = 0.3 # eV
153 Twarm = 2. # eV
154 Thot = hesel:Ti0 # eV
155
156 sigma = 3.53e-20 # m^2
157 m = 2*1.67e-27 # kg
158 e = 1.60e-19 # C
159
160 Dcold = 20. # m^2s^-1
161 Dwarm = 53. # m^2s^-1
162 Dhot = 412. # m^2s^-1
163
164 v_cold_x = -758. # ms^-1

```

```

165 v_warm_x = -2000. # ms^-1
166 v_hot_x = -2800. # ms^-1
167
168 inputBCs = false
169 wallAbsorbctionFraction = 0.2
170
171 floor_neutral_profiles = false
172 floor_neutral_density = 1e-9
173 neutral_floor_time = 50
174
175
176 [init_neutrals]
177 oci_proxy = 1e-19/1e-27
178 rhos_proxy = sqrt(1/oci_proxy)
179 k_proxy = 1e-14*hesel:n0/oci_proxy
180
181 NcoldConst = 1/(rhos_proxy*oci_proxy*hesel:n0)*neutrals:nColdFlux/(sqrt(k_proxy*
    init_n:n_bg/neutrals:D_cold) * exp(sqrt(k_proxy*init_n:n_bg/neutrals:D_cold)*
    mesh:Lx*hesel:x_wall))
182
183 Ncold_SOL = NcoldConst*exp(sqrt(k_proxy*init_n:n_bg/neutrals:D_cold)*(hesel:xr -
    init_n:x_shift_n))
184 Ncold_EDGE = NcoldConst*exp(sqrt(k_proxy*init_n:n_inner/neutrals:D_cold)*(hesel:
    xr - init_n:x_shift_n))
185 Ncold_INIT = exp((x-1)/0.1)
186
187 Nwarm_INIT = 1e-3
188
189 Nhot_INIT = 1e-3
190
191 # Neutral profiles
192 [lnNcold]
193 scale = 1.0
194 function = log(init_neutrals:Ncold_INIT)
195
196 bndry_xin = neumann
197 bndry_xout = none
198
199 [lnNwarm]
200 scale = 1.0
201 function = log(init_neutrals:Nwarm_INIT)
202
203 bndry_xin = neumann
204 bndry_xout = none
205
206 [lnNhot]

```

```

207 scale    = 1.0
208 function = log(init_neutrals:Nhot_INIT)
209
210 bndry_xin  = neumann
211 bndry_xout = none
212
213 #=====
214 #Include kinetic neutrals
215 #=====
216 [kinetic_neutrals]
217
218 H_atom_injection_rate = 0
219 H_atom_N_max = 1000000
220 H_molecule_temperature = 0.3 #Injection temperature for molecules
221 H_atom_temperature = 1
222 H_molecule_injection_rate = 250*1e+10 #Injected supermolecules per second
223 phys_injection_rate_molecules = 150e+19
224 H_molecule_N_max = 3000000 #Maximum number of molecules.
225 flag_initialized = 0 #If 0 run initializer, else load initiated objects from files
226 dt_min = 1.0 #Minimum timestep for kinetic neutral model
227 step_length = 1.0 #Step length for the kinetic neutral model in oci units
228 r_std_blur = 2.0 #standard deviation for blur expressed in rhos.
229 init_source_time = 1.0 #Time length over which the sources for the first timestep
    in plasma sim is calculated
230 wallAbsorbtionFractionAtom = 0.2 # Fraction absorbed
231 wallAbsorbtionFractionMolecule = 0.2
232
233
234 #=====
235 #=====
236 [All] # default setttings for variable boundary conditions
237 scale = 0.0
238 bndry_all = neumann
239 #=====
240 #=====
241 [init_n] # initial profile for density, n_LCFS = n0
242 scale = 1.0
243 n_inner = 1.5 # inner boundary value for density
244 n_bg = 0.1 # background density
245 x_shift_n = 2.93794 # shift x in order to set n_lcfs = n0, for n_inner=1.5, n_bg
    =0.1 and step_width=10.
246 step_width = 10. # step width for tanh function
247
248 function = 0.5 * (n_inner - n_bg) * (1 - tanh( (hesel:xr - x_shift_n)/step_width))
    + n_bg
249

```



```

250 [init_pe] # initial profile for electron pressure, pe_LCFS = n0*Te0
251 scale = 1.0
252 pe_inner = 9.0 # inner boundary value for electron pressure
253 pe_bg = 0.01 # background electron pressure
254 x_shift_pe = -3.51585 # shift x in order to set pe_lcfs = n0*Te0, for pe_inner=9,
    pe_bg=0.01 and step_width=10.
255 step_width = 10. # step width for tanh function
256
257 line_slope = (sqrt(pe_inner)-1)/hesel:edge_width
258 edge_prof = 1 - line_slope * sigma_closed:function * hesel:xr
259 function = (0.5 *(sqrt(pe_inner) - pe_bg) * (1 - tanh( (hesel:xr - x_shift_pe)/
    step_width))) + pe_bg) * edge_prof
260
261 [init_pi] # initial profile for ion pressure, pi_LCFS = n0*Ti0
262 scale = 1.0
263 pi_inner = 9.0 # inner boundary value for ion pressure
264 pi_bg = 0.01 # background ion pressure
265 x_shift_pi = -3.51585 # shift x in order to set pi_lcfs = n0*Ti0, for pi_inner=8,
    pi_bg=0.01 and step_width=10.
266 step_width = 10. # step width for tanh function
267
268 line_slope = (sqrt(pi_inner)-1)/hesel:edge_width
269 edge_prof = 1 - line_slope * sigma_closed:function * hesel:xr
270 function = (0.5 *(sqrt(pi_inner) - pi_bg) * (1 - tanh( (hesel:xr - x_shift_pi)/
    step_width))) + pi_bg) * edge_prof
271
272 [seed_n]
273 scale = 1.0
274 pert_amp = 0.001
275 pert_size = 5
276 xaxis = mesh:xl-hesel:force_width
277 zaxis = mesh:zl-mesh:Lz/2
278 seed = 161803
279 function = pert_amp*(5 + mixmode(z, seed)*mixmode(2*pi*x, seed)) # pert_amp*(1 +
    sin(z^2/pi)*sin(10*2*pi*x)) # pert_amp*exp(-(xaxis*xaxis + zaxis*zaxis)/(2*
    pert_size*pert_size))
280 #=====
281 #=====
282 [lnn] # initial profile for lnn field
283 scale = 1.0
284 bndry_xin = dirichlet_o2(log(init_n:n_inner))
285 bndry_xout = neumann_o2(0.)
286 function = log(init_n:function+seed_n:function)
287
288 [lnpe] # initial profile for lnpe field
289 scale = 1.0

```

```

290 bndry_xin = dirichlet_o2(log(init_pe:pe_inner))
291 bndry_xout = neumann_o2(0.)
292 function = log(init_pe:function)
293
294 [lnpi] # initial profile for lnpi field
295 scale = 1.0
296 bndry_xin = dirichlet_o2(log(init_pi:pi_inner))
297 bndry_xout = neumann_o2(0.)
298 function = log(init_pi:function)
299
300 [vort] # initial profile for vort field
301 scale = 1.0
302 bndry_xin = dirichlet_o2(0.)
303 bndry_xout = neumann_o2(0.)
304 function = 0.
305 #=====
306 #=====
307 [sigma_open] # step function for open field lines
308 scale = 1.0
309 function = 0.5 * (1+tanh(hesel:xr/hesel:d_lcfs)) + hesel:wall_amp*0.5 * (1+tanh((
hesel:xr-hesel:sol_width)/hesel:d_wall))
310
311 [sigma_closed] # step function for closed field lines
312 scale = 1.0
313 function = 0.5 * (1-tanh(hesel:xr/hesel:d_lcfs))
314
315 [sigma_force] # step function for forced profiles
316 scale = 1.0
317 function = 0.5 * (1-tanh((mesh:x1-hesel:force_width)/hesel:d_force))
318 #=====
319 #=====
320 [fast_output]
321 type = monitor
322 flushfrequency = 100
323
324 xpos0 = 0.2
325 ypos0 = 0
326 zpos0 = 0.5
327
328 xpos1 = 0.3
329 ypos1 = 0
330 zpos1 = 0.5
331
332 xpos2 = 0.4
333 ypos2 = 0
334 zpos2 = 0.5

```

```
335
336 xpos3 = 0.5
337 ypos3 = 0
338 zpos3 = 0.5
339
340 xpos4 = 0.6
341 ypos4 = 0
342 zpos4 = 0.5
343
344 xpos5 = 0.7
345 ypos5 = 0
346 zpos5 = 0.5
347
348 xpos6 = 0.8
349 ypos6 = 0
350 zpos6 = 0.5
```

8.2 Opsætning af BOUT++ og BOUT-HESEL

Listing 2: Gennemgang af opsætning og installation af BOUT++ og BOUT-HESEL

```
1 #####
2 #Refresh alt#
3 #####
4 rm -rf BOUT/*
5
6 #####
7 #Sæt env#
8 #####
9 qrsh
10 #Vent til du er kommet ind på noden
11
12 export OMP_NUM_THREADS=16
13 module purge
14 module load mpi/5.0.3-gcc-13.3.0-binutils-2.42
15 module load fftw3/3.3.10-openmpi-5.0.3-gcc-13.3.0
16 module load netcdf-c/4.9.3-hdf5-1.14.6-intel-2021-update4
17 module load intel/2021.4.0
18
19 #####
20 #Load byg dev#
21 #####
22 cd BOUT
23
24 git clone https://github.com/boutproject/BOUT-dev.git
25 cd BOUT-dev
```

```
26 git checkout v4.3.2
27 ./configure
28 make -j16
29
30
31 #####
32 #Load byg hesel#
33 #####
34 cd BOUT
35
36 git clone --recursive git@github.com:PPFE-Turbulence/BOUT-HESEL.git
37 cd BOUT-HESEL
38
39 git config --file=.gitmodules submodule.HESEL-Common.url git@github.com:PPFE-
    Turbulence/HESEL-Common.git
40 git submodule sync
41 git submodule update --init --recursive
42
43 ln -s HESEL-Common/BoutFastOutput BoutFastOutput
44 ln -s HESEL-Common/HeselParameters HeselParameters
45 ln -s HESEL-Common/Neutrals Neutrals
46 ln -s HESEL-Common/Parallel Parallel
47 make -j16
48
49 #####
50 #Kør simulation#
51 #####
52 cd BOUT-HESEL
53
54 module purge
55 module load mpi/5.0.3-gcc-13.3.0-binutils-2.42
56 module load fftw3/3.3.10-openmpi-5.0.3-gcc-13.3.0
57 module load netcdf-c/4.9.3-hdf5-1.14.6-intel-2021-update4
58 module load intel/2021.4.0
59
60 mpirun -np 4 ./hesel -d data
```

9 Appendix B

9.1 Anvendelse af Boltzmann ligning

Givet 2D rummet, med fordelingsfunktionen:

$$\begin{aligned}\beta &= (1, 2) \\ \mathbf{x} &= (x_1, x_2) \\ \mathbf{v} &= (v_1, v_2) \\ f_a(t, x_1, x_2, v_1, v_2) &= tx_1^2 + x_2v_1 + v_2^2\end{aligned}$$

Her ses bort fra kraftleddet ved at sætte $\frac{F_{a\beta}}{m_a} = \alpha$,
Ligningen er givet ved 12. For Einstein summation over β gælder:

$$\partial_{x_\beta}(v_\beta f_a) = \partial_{x_1}(v_1 f_a) + \partial_{x_2}(v_2 f_a).$$

Herved kan de afledte findes ved:

- Tidsafledte

$$\partial_t f_a = \partial_t(tx_1^2 + x_2v_1 + v_2^2) = x_1^2.$$

- Rumleddet

$$\partial_{x_\beta}(v_\beta f_a) = \partial_{x_1}(v_1 f_a) + \partial_{x_2}(v_2 f_a) = 2v_1tx_1 + v_1v_2$$

- Hastighedsleddet

$$\partial_{v_\beta}(\alpha f_a) = \partial_{v_1}(\alpha f_a) + \partial_{v_2}(\alpha f_a) = \alpha x_2 + 2\alpha v_2$$

- Samling med kollisionsleddet

$$x_1^2 + 2tx_1v_1 + v_1v_2 + \alpha x_2 + 2\alpha v_2 = C_a$$